

Yes, absolutely. This is actually a very realistic scenario.

Since this is a **Proof of Concept (POC)** and **not production**, it is reasonable to hard-code the connection string temporarily **only for testing**, provided you remove it before the code is promoted to production. For production, moving the secret to Azure Key Vault (or at least Azure DevOps secret variables) is the right approach.

## What we'll build

We'll create this flow in your lab:



This will demonstrate to your customer that the migration works end-to-end.

## Lab Requirements

On your Windows machine, we'll have:

| Component                      | Status               |
|--------------------------------|----------------------|
| Azure DevOps Self-hosted Agent | ✓ Already configured |
| Docker Desktop                 | ✓ Installed          |
| Java                           | ✓ Installed          |
| Maven                          | ✓ Installed          |

| Component             | Status              |
|-----------------------|---------------------|
| .NET SDK 8            | To verify           |
| PostgreSQL            | We'll run in Docker |
| GitHub Repository     | ✓ Available         |
| Azure DevOps Pipeline | We'll create/update |

## Phase 1 – Start PostgreSQL in Docker

Pull the PostgreSQL image:

❏ Bash

```
docker pull postgres:16
```



Run PostgreSQL:

❏ Bash

```
docker run -d ^  
  --name postgres-lab ^  
  -e POSTGRES_USER=postgres ^  
  -e POSTGRES_PASSWORD=Password@123 ^  
  -e POSTGRES_DB=EmployeeDB ^  
  -p 5432:5432 ^  
  postgres:16
```



Verify:

❏ Bash

```
docker ps
```



Expected:

```
postgres-lab  
Up  
0.0.0.0:5432->5432
```



## Phase 2 – Verify PostgreSQL

Enter the container:

</> Bash



```
docker exec -it postgres-lab bash
```

Login:

</> Bash



```
psql -U postgres
```

Show databases:

</> SQL



```
\l
```

Connect:

</> SQL



```
\c EmployeeDB
```

List tables:

</> SQL



```
\dt
```

Initially:

```
No relations found.
```



Perfect—that means the migration will create the table.

---

## Phase 3 – Modify Program.cs (Temporary)

For the POC, you can temporarily replace:

</> C#



```
Environment.GetEnvironmentVariable("PG_CONNECTION_STRING")
```

with:

</> C#



```
var connectionString =  
"Host=localhost;Port=5432;Database=EmployeeDB;Username=postgres;Password=Password@123";
```

Again, **only for the demo**.

---

## Phase 4 – Test Locally

Restore packages:

</> Bash



```
dotnet restore
```

Build:

</> Bash



```
dotnet build
```

Run:

</> Bash



```
dotnet run
```

Expected output:

```
Connected to PostgreSQL server.
```



```
Table 'employee' ensured.
```

```
Inserted Bala
```

```
Inserted Anitha
```

```
Migration completed successfully.
```

---

## Phase 5 – Verify the Table

Inside PostgreSQL:

</> SQL



```
\dt
```

Expected:

employee



View the data:

</> SQL



```
SELECT * FROM employee;
```

Expected:

| employee_id | first_name | last_name |
|-------------|------------|-----------|
| 1           | Bala       | Kuppusamy |
| 2           | Anitha     | Raman     |

This is what you'll show the customer.

## Phase 6 – Azure DevOps Pipeline

We'll build the project and then run it.

Example pipeline:

</> YAML



```
trigger:
- main

pool:
  name: Default

steps:

- task: UseDotNet@2
  inputs:
    packageType: sdk
    version: '8.0.x'

- script: dotnet restore
  displayName: Restore
```

```
- script: dotnet build --configuration Release
  displayName: Build

- script: dotnet run
  displayName: Run Migration
```

Since the program contains the temporary hard-coded connection string, no pipeline variables are required for this POC.

---

## Phase 7 – Demonstrate to the Customer

Show:

### 1. Docker Container

❏ Bash

```
docker ps
```



Customer sees PostgreSQL running.

---

### 2. Azure DevOps Pipeline

Successful pipeline:

✓ Restore

✓ Build

✓ Run Migration



### 3. PostgreSQL

Run:

❏ SQL

```
SELECT * FROM employee;
```



Customer sees:



---

## 4. Re-run the Pipeline

Run the pipeline a second time.

Show that:

- The table is **not recreated** because of `CREATE TABLE IF NOT EXISTS`.
- Duplicate rows are **not inserted** because of `ON CONFLICT(email) DO NOTHING`.

This demonstrates that the migration is **idempotent**, which is a strong point to highlight.

---

## After Customer Approval

Once the customer confirms the solution works, we can make it production-ready by:

- Removing the hard-coded connection string.
  - Storing the connection string in **Azure Key Vault**.
  - Retrieving it securely from the Azure DevOps pipeline.
  - Separating **Build** and **Deploy** stages.
  - Publishing the compiled application as a pipeline artifact.
  - Adding deployment approvals for production.
- 

## My Recommendation

I suggest we **don't rush into creating the pipeline immediately**. Let's do it in the same order a real DevOps team would:

1. Verify the .NET project builds locally.
2. Start PostgreSQL in Docker.
3. Run the migration locally and confirm the table is created.
4. Create the Azure DevOps pipeline.
5. Run the same migration through the pipeline.
6. Verify the database again after the pipeline completes.

That way, if something goes wrong, we'll know exactly which step needs attention instead of debugging multiple moving parts at once.

We'll work through each step together until you have a complete, customer-ready demonstration.